Python Engineering - Programming Theory Guide

CLASS 16 - Testing & Project Structure

Testing answers a concrete question: **how do I know the program still does what I believe it does?** This guide connects that question with engineering validation, numerical methods, maintenance and software structure.

1. Running is not verification

```
PROGRAM RUNS
    !=
PROGRAM IS CORRECT
```

A program can finish without exceptions and still produce a physically incorrect answer. Tests compare behavior against explicit expectations.

2. A test as an experiment

```
known initial conditions -> system under test -> observed result -> reference comparison
```

This resembles a laboratory test: prepare conditions, apply an action, measure and compare.

3. Arrange - Act - Assert as an engineering method

Testing	Engineering analogue
Arrange	Prepare initial conditions/configuration
Act	Run calculation/simulation/test
Assert	Compare observation with reference/criterion

4. Test oracle

The expected result acts as an oracle: a reference that lets us decide whether behavior is correct.

```
Euler reference after dt=0.1:
[102.0, 19.019, 0.1]
```

An oracle may come from an analytical solution, hand calculation, validated dataset, standard or previously verified result.

5. Regression

```
version A -> test passes
modify code
version B -> same test fails
              ^ regression detected
```

Tests turn previous knowledge into an automatic alarm against regressions.

6. What deserves a test?

Type	EX24 example
Nominal behavior	VerticalState returns initial state
Numerical update	Euler produces expected state
Module integration	Save + load preserves trajectory
Error behavior	Missing file raises FileNotFoundError

7. Unit-like vs integration-like tests

```
UNIT-LIKE
VerticalState.euler_step()

INTEGRATION-LIKE
save CSV -> disk -> load CSV
```

EX24 contains both styles to show that testing is not one single kind of check.

8. Deterministic and repeatable tests

The same test under the same conditions should produce the same result. Hidden external state makes PASS/FAIL unreliable.

```
better: test creates its own input and cleans it afterwards
```

9. Test independence

If test B only works after test A, execution order becomes a hidden dependency.

```
test A: arrange -> act -> assert
test B: arrange -> act -> assert
```

10. Numerical testing and tolerances

Floating-point values have finite representation, so mathematically equivalent calculations may differ slightly in binary representation.

np.allclose asks whether values are sufficiently close within tolerances.

11. Tolerance is not unlimited permissiveness

Tolerance should match problem scale and required precision. EX24 uses simple defaults; GNSS, orbital dynamics or CFD may require domain-specific limits.

12. Testing errors is testing requirements

```
invalid input -> expected exception -> PASS
```

If the contract requires FileNotFoundError for a missing file, that exception is part of correct behavior.

13. PASS, FAIL and ERROR mean different things

```
PASS  -> expected behavior observed
FAIL  -> code ran, assertion disagreed
ERROR -> test could not reach normal assertion path
```

Separating them speeds diagnosis.

14. Test runner as supervisor

```
run_tests.py -> test A / test B / test C -> summary
```

The runner coordinates tests just as main.py coordinates application behavior.

15. Separation of responsibilities

```
vertical_state.py -> physics/model
file_io.py        -> persistence
main.py           -> application orchestration
tests/            -> verification
run_tests.py      -> test orchestration
```

Clear responsibilities reduce side effects and make failures easier to locate.

16. Why a tests/ folder?

It communicates that verification code is separate from normal application behavior and scales better as tests grow.

17. Structure as documentation

```
project/
|-- main.py
|-- model.py
|-- file_io.py
|-- data/
`-- tests/
```

A good structure explains the project before anyone reads implementation details.

18. GitHub-oriented structure

Repository visitors want to understand what the project does, how to run it and how to verify it. Separating application code and tests improves readability and engineering credibility.

> Later we can add README, requirements, .gitignore and automated CI testing; EX24 prepares that foundation.

19. Test value is not test count

A small set of critical tests can provide more value than many redundant tests.

```
initial state + Euler update + round trip + expected failure
```

20. What good tests should avoid

Avoid	Why
Depending on test order	Fragile
Hidden manually prepared data	Not reproducible
Checking too many behaviors at once	Hard diagnosis
Swallowing all exceptions	False PASS risk
Reimplementing all program logic in the test	May duplicate same bug

21. Testing and numerical methods

Testing becomes even more important in the next course block because numerical algorithms may produce plausible but incorrect results.

```
root finding
integration
differentiation
ODE solvers
statistics
-> validate against known cases
```

> Before trusting complex algorithms, test them on simple problems with known solutions.

22. Testing engineering software

Area	Example test
Propulsion	Known thrust under reference conditions
GNSS	Position/pseudorange against validated dataset
Astrodynamics	Energy conservation in ideal case
Dynamics	State after a known time step
Data processing	File round-trip and format

23. Final mental model

```
CODE
 -> TESTS encode expectations
 -> PASS / FAIL / ERROR

CHANGE CODE
 -> RUN TESTS AGAIN
 -> confidence against regressions
```

24. Summary

```
TEST = executable expectation
ARRANGE = prepare
ACT = execute
ASSERT = compare
REGRESSION = old behavior broken by a new change
REPEATABLE = same setup -> same result
INDEPENDENT = no hidden dependence on other tests
STRUCTURE = responsibilities visible in folders/files
```